

Autoregressive vs. Masked Diffusion Language Models: A Controlled Comparison

Caio Vicentino
Independent Researcher
github.com/caiovicentino/arche

Abstract

We present a controlled empirical comparison between autoregressive (AR) and masked diffusion (MDLM) language models. Both models are trained on identical data (50M tokens from TinyStories), identical compute budget (20,000 steps, batch size 32, sequence length 512), and identical hardware (NVIDIA H100 80GB), isolating the generation paradigm as the sole variable. We report three findings. First, both paradigms achieve comparable training throughput ($\sim 50\text{K}$ tokens/second), with MDLM requiring only 4.7% more wall-clock time, countering the perception that diffusion training is substantially more expensive. Second, AR converges faster and begins overfitting by step 14,000, while MDLM converges more slowly and is still improving at step 20,000, suggesting different compute-optimal training regimes. Third, qualitative analysis of generated text reveals a diversity–fluency trade-off: AR produces fluent but structurally repetitive outputs (identical formulaic openings across samples), while MDLM generates more diverse narratives with varied characters, structures, and starting points, at the cost of occasional grammatical inconsistencies. We note that validation losses across paradigms are not directly comparable, as they measure different objectives (next-token prediction vs. masked token prediction), and that the diversity observation is preliminary and requires quantitative validation. All code, trained checkpoints, and data pipelines are released for reproducibility.

1 Introduction

Language models have been overwhelmingly autoregressive: tokens are generated left-to-right, each conditioned on all preceding tokens. This paradigm underpins GPT [Radford et al., 2019], LLaMA [Touvron et al., 2023], and virtually all production language models. Its strengths are well understood—efficient training via teacher forcing, straightforward sampling, and strong perplexity—but so are its limitations: sequential generation that cannot revise earlier tokens, and a strong left-to-right bias that can produce repetitive outputs when the most probable prefix dominates.

A fundamentally different paradigm has recently emerged: diffusion-based language models. Adapted from the continuous diffusion framework that revolutionized image generation [Ho et al., 2020, Song et al., 2021], these models generate text by starting from a fully corrupted (masked) sequence and iteratively denoising it. Masked Diffusion Language Models (MDLM; Sahoo et al. 2024) use a learned masking schedule to corrupt text during training and reverse the process during generation. SEDD [Lou et al., 2024] achieved ICML 2024 Best Paper with substantially improved generative perplexity over GPT-2. LLaDA [Nie et al., 2025] demonstrated competitive performance at 8B parameters, matching LLaMA3 on downstream benchmarks. Gemini Diffusion [Google DeepMind, 2025] demonstrated fast generation at commercial quality.

Despite this rapid progress, controlled head-to-head comparisons remain scarce. Existing studies either compare at different scales, use different datasets, or employ different compute budgets, making it difficult to attribute observed differences to the generation paradigm itself rather than confounding variables.

This work. We train an autoregressive Transformer and a masked diffusion Transformer on *exactly the same* data, compute budget, and hardware. The only differences are those inherent to each paradigm: causal vs. bidirectional attention, next-token vs. masked-token objective, and the MDLM’s timestep conditioning module. This controlled setup allows us to isolate the effect of the generation paradigm on training dynamics, convergence behavior, and generation quality.

We acknowledge upfront that our experiments are at small scale (123–163M parameters, 50M tokens) and that our generation quality analysis is qualitative rather than quantitative. Our goal is not to declare a winner, but to provide a reproducible baseline comparison that highlights the distinct trade-offs of each paradigm.

1.1 Contributions

1. **Controlled experimental setup:** AR and MDLM trained with identical data, steps, batch size, sequence length, model dimensions, optimizer, and hardware (Table 1). To our knowledge, this is the most tightly controlled small-scale comparison of these paradigms to date.
2. **Training dynamics analysis:** We provide step-by-step convergence curves (Table 3) showing that AR converges faster but overfits earlier, while MDLM converges slowly but steadily, suggesting different compute-optimal training lengths.
3. **Throughput parity:** We find that MDLM training is only 4.7% slower than AR in wall-clock time (113.0 vs. 107.9 minutes), countering the common assumption that diffusion training is substantially more expensive.
4. **Preliminary diversity–fluency observation:** Qualitative inspection suggests AR and MDLM occupy different points on a diversity–fluency trade-off, motivating future quantitative studies.
5. **Full reproducibility:** All code, data pipelines, trained checkpoints (PyTorch), and evaluation scripts released at <https://github.com/caiovicentino/arche>.

2 Related Work

2.1 Autoregressive Language Models

The autoregressive Transformer [Vaswani et al., 2017, Radford et al., 2019] remains the dominant language modeling paradigm. Scaling laws [Kaplan et al., 2020, Hoffmann et al., 2022] have established predictable relationships between model size, data, and performance. Recent architectural innovations include Multi-Head Latent Attention for KV-cache compression [DeepSeek-AI, 2024], Gated Linear Attention for efficient training [Yang et al., 2024], and State Space Models for linear-time inference [Gu & Dao, 2023]. All of these operate within the autoregressive framework.

2.2 Diffusion Language Models

Adapting diffusion to discrete text has been an active area. D3PM [Austin et al., 2021] introduced discrete diffusion with transition matrices. MDLM [Sahoo et al., 2024] simplified this to a masking-based framework with a cosine schedule, achieving strong results with standard Transformer backbones. SEDD [Lou et al., 2024] used score-based estimation for discrete distributions, earning ICML 2024 Best Paper. LLaDA [Nie et al., 2025] scaled masked diffusion to 8B parameters, demonstrating competitive downstream performance with LLaMA3. Gemini Diffusion [Google DeepMind, 2025] achieved commercial-grade generation with 5× speed improvements.

Table 1: Controlled experimental setup. All shared variables are identical; differences arise solely from the paradigm.

Variable	AR	MDLM
<i>Shared (identical)</i>		
Data	TinyStories [Eldan & Li, 2023], 50M tokens	
Steps	20,000	
Batch size	32	
Sequence length	512	
d_{model}	768	
Layers	12	
Heads	12	
FFN dim	2,048	
Optimizer	AdamW (lr = 3e-4, wd = 0.01)	
Hardware	NVIDIA H100 80GB	
<i>Paradigm-specific (inherent differences)</i>		
Attention mask	Causal	Bidirectional
Training objective	Next-token prediction	Masked token prediction
Noise schedule	N/A	Cosine
Timestep conditioning	N/A	Sinusoidal embedding
Parameters	123.6M	162.7M

The key architectural difference from AR models is that diffusion models use *bidirectional* attention (each position can attend to all unmasked positions) and condition on a noise level or timestep. This bidirectional context is both a strength (richer conditioning) and a difference that complicates direct comparison.

2.3 Existing Comparisons

Prior comparisons between AR and diffusion language models exist but are not tightly controlled. Sahoo et al. [2024] compare against GPT-2 but at different parameter counts and with different training data. Nie et al. [2025] compare against LLaMA3 but with different training corpora and compute budgets. Lou et al. [2024] report perplexity improvements over GPT-2 but using a different model architecture. Our work fills the gap of a *same-data, same-budget, same-hardware* comparison, albeit at small scale.

3 Experimental Setup

3.1 Controlled Variables

Table 1 summarizes the experimental design. All shared hyperparameters—data, training steps, batch size, sequence length, model dimensions, number of layers, number of heads, FFN dimension, optimizer, and hardware—are identical between the two models.

3.2 Parameter Count Difference

The MDLM model has 31.6% more parameters than the AR model (162.7M vs. 123.6M). This difference is not an oversight but reflects inherent architectural requirements:

- **Bidirectional attention:** The MDLM uses full (non-causal) attention, which does not change the parameter count but does change the computation pattern.

- **Timestep conditioning:** The MDLM requires a timestep embedding module (sinusoidal embedding followed by projection layers) to condition on the noise level, adding parameters with no AR counterpart.
- **Output head:** The MDLM predicts tokens at all masked positions simultaneously, potentially requiring a slightly different output projection structure.

We chose not to artificially reduce the MDLM to match the AR parameter count, as this would compromise the MDLM design. This difference should be considered when interpreting results: the MDLM has greater model capacity, which could favor it in principle, yet it converges more slowly in practice.

3.3 Autoregressive Model

The AR model is a standard decoder-only Transformer with causal (unidirectional) attention. Each position attends only to itself and all preceding positions. The training objective is standard next-token prediction: given a sequence $(x_1, \dots, x_T)$, the model minimizes

$$\mathcal{L}_{\text{AR}} = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}) \quad (1)$$

We use learned positional embeddings and pre-norm (LayerNorm before attention and FFN).

3.4 Masked Diffusion Language Model (MDLM)

The MDLM follows [Sahoo et al. \[2024\]](#). The architecture is a bidirectional Transformer—identical to the AR model except that the causal mask is removed, allowing each position to attend to all other positions. Additionally, a timestep embedding conditions the model on the current noise level.

Training. For each training example, we sample a random timestep $t \sim U(0, 1)$ and mask tokens independently with probability

$$\gamma(t) = 1 - \cos^2\left(\frac{\pi t}{2}\right) \quad (\text{cosine schedule}) \quad (2)$$

Masked tokens are replaced with a special [MASK] token. The model predicts the original token at each masked position, with cross-entropy loss computed only at masked positions:

$$\mathcal{L}_{\text{MDLM}} = -\mathbb{E}_{t, \mathbf{m}} \sum_{i: m_i=1} \log p_{\theta}(x_i \mid \tilde{\mathbf{x}}, t) \quad (3)$$

where $\tilde{\mathbf{x}}$ is the masked input and $\mathbf{m}$ is the binary mask.

Generation. Starting from a fully masked sequence of length L , we iteratively unmask tokens over $S = 100$ steps. At each step s , the model predicts all masked positions; we select the $\lfloor L/S \rfloor$ most confident predictions (highest predicted probability) and unmask them. We apply temperature annealing (starting high for diversity, decreasing for coherence) and a repetition penalty to reduce degenerate outputs.

Table 2: AR vs. MDLM training comparison on H100 80GB (20K steps, TinyStories 50M tokens).

Metric	AR	MDLM
Parameters	123.6M	162.7M
Training time	107.9 min	113.0 min
Throughput	50,620 tok/s	48,343 tok/s
Best val loss	1.589	3.412
Best checkpoint	Step 14,000	Step 20,000

Val losses use different objectives and are not cross-comparable.

4 Results

4.1 Training Efficiency

Both paradigms achieve comparable training throughput on the H100 (Table 2). AR trains at 50,620 tokens/second; MDLM at 48,343 tokens/second (4.5% slower). Total wall-clock time differs by only 5.1 minutes (107.9 vs. 113.0 minutes for 20K steps). This near-parity is notable: it suggests that the computational overhead of diffusion training (sampling timesteps, applying masks, computing loss only at masked positions) is modest relative to the dominant cost of the forward and backward passes through the Transformer.

4.2 Convergence Dynamics

Table 3 and Figure 1 show validation loss over training. The two loss functions measure different quantities—next-token prediction (AR) vs. masked token prediction with bidirectional context (MDLM)—so the absolute values are not cross-comparable. However, the *convergence patterns* are directly comparable and reveal a clear difference:

- **AR converges fast, overfits early.** Validation loss reaches its minimum of 1.589 at step 14,000, then rises to 1.622 by step 20,000—a clear sign of overfitting on 50M tokens at this model size.
- **MDLM converges slowly, keeps improving.** Validation loss decreases monotonically throughout training, reaching 3.412 at step 20,000 with no sign of plateauing. This suggests MDLM would benefit from substantially longer training.

This asymmetry has practical implications: AR models at this scale need early stopping or regularization after ~ 14 K steps on 50M tokens, while MDLM models can productively train for longer. Whether this reflects a fundamental property of the paradigm or simply the effect of the larger parameter count (Section 3.2) is an open question.

4.3 Generation Quality: Qualitative Analysis

We present a preliminary qualitative comparison of generated text. We stress that this analysis is *illustrative, not conclusive*: it is based on a small number of samples and does not include quantitative diversity or fluency metrics. Rigorous evaluation with Self-BLEU [Zhu et al., 2018], Distinct- n [Li et al., 2016], or MAUVE [Pillutla et al., 2021] over hundreds of samples is an important direction for future work.

AR samples. We generate three independent samples from the AR model using nucleus sampling ($p = 0.9$, temperature 0.8):

Table 3: Validation loss at selected checkpoints. Each column measures a different objective; read vertically for convergence trends, not horizontally.

Step	AR Val Loss	MDLM Val Loss
2,000	2.660	3.952
4,000	2.047	3.841
6,000	1.837	3.585
8,000	1.726	3.528
10,000	1.665	3.509
12,000	1.633	3.527
14,000	1.589	3.491
16,000	1.603	3.455
18,000	1.627	3.419
20,000	1.622	3.412

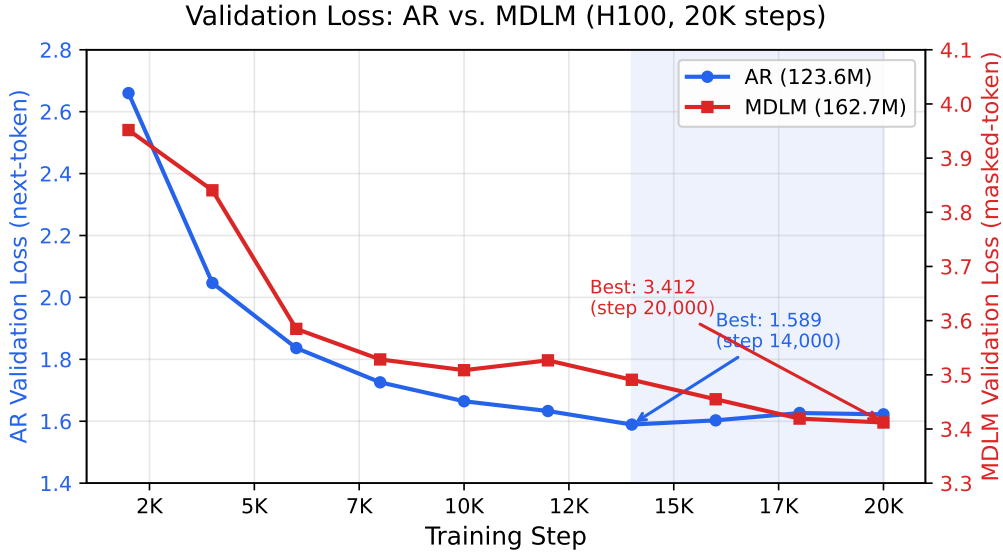


Figure 1: Convergence comparison. Dual y -axes reflect different loss functions; the shape of each curve (not the absolute value) is the relevant comparison. AR overfits after step 14K (blue shaded region); MDLM continues improving monotonically.

AR-1: “Once upon a time, there was a little girl named Jane. She loved to jump in the pool. She would splash and jump and have fun...”

AR-2: “Once upon a time, there was a little girl named Lily. She lived in a small village near a big mountain. One day, Lily asked her mom, ‘Can we go to the mountain...’”

AR-3: “Once upon a time, there was a little girl named Lily. She liked to play with her toys and eat cereal with a spoon.”

All three samples begin with the identical formulaic opening (“Once upon a time, there was a little girl named X ”). The text is grammatically fluent and narratively coherent, but structurally repetitive.

MDLM samples. We generate three independent samples from the MDLM using confidence-based unmasking (100 steps, temperature annealing 1.2→0.5):

MDLM-1: “They sit on the grass and eat their lunch. They are very happy and full of energy! They have such great time playing together...”

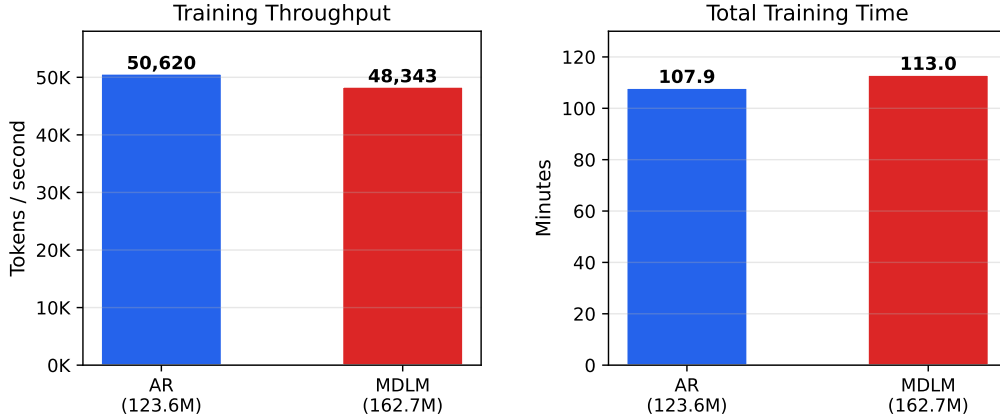


Figure 2: Training throughput and wall-clock time. MDLM achieves 95.5% of AR throughput, requiring only 5.1 additional minutes over 20K steps.

MDLM-2: “Lucy’s birthday?” They say... ‘Happy Birthday!’ From writing on paper, they always made sure to say ‘please’... When they were all done, Lucy felt so happy that She said loudly ‘Thank you’, then hugged my mom.”

MDLM-3: “‘Hi I’m Timmy! Can we play with you?’ she asked. ‘Sure, let’s go!’ said Timmy as he drove his toy car around the playground. He had so much fun playing on the swings until dark...”

MDLM samples show: (a) different narrative starting points (none begins with “Once upon a time”), (b) varied character names and situations, (c) natural dialogue, and (d) more creative scenarios. However, they exhibit occasional grammatical inconsistencies (“They have such great time”, capitalized “She” mid-sentence) and punctuation artifacts.

Interpretation. The observed diversity difference is plausibly architectural. In AR generation, the model commits to tokens sequentially: once a high-probability prefix is produced (e.g., “Once upon a time”), the conditional distribution over remaining tokens is heavily constrained by the left-to-right dependency. In MDLM generation, all positions begin as [MASK] and are revealed in parallel based on model confidence. The model can generate the middle of a narrative before the beginning, avoiding the “mode collapse to the most probable prefix” phenomenon.

This parallels findings in image generation: autoregressive pixel models produce sharp but low-diversity images, while diffusion models produce higher-diversity outputs [Ho et al., 2020]. Our observations suggest this diversity–fidelity trade-off may extend to language, though we emphasize this hypothesis requires rigorous quantitative validation.

5 Discussion

5.1 Throughput Parity Is Surprising

A common concern about diffusion language models is training cost. Our finding that MDLM trains at 95.5% of AR throughput (48,343 vs. 50,620 tokens/second) suggests that the per-step overhead of diffusion training—sampling timesteps, constructing masks, computing loss only at masked positions—is negligible relative to the Transformer forward/backward pass. At larger scales where the Transformer computation dominates even more, the gap may shrink further.

This does *not* mean the two paradigms have equal sample efficiency. AR reached its best validation loss at 14K steps; MDLM was still improving at 20K steps. If MDLM requires, say, 2–3× more steps to converge, the total training cost would be correspondingly higher despite

similar per-step throughput. Characterizing the step-efficiency gap is an important question for future work.

5.2 Different Convergence Regimes

The divergent overfitting behavior—AR overfits at 14K steps while MDLM continues improving at 20K—is consistent with a capacity-utilization hypothesis. AR sees each token in a fixed left-to-right context; it can memorize common patterns quickly. MDLM sees each token in a randomly masked context that varies every epoch; this implicit data augmentation may act as a regularizer, slowing convergence but reducing overfitting.

Alternatively, the difference may simply reflect the parameter count gap (Section 3.2): the MDLM’s larger model may need more steps to fit the data. Disentangling these explanations requires experiments with matched parameter counts, which we leave for future work.

5.3 The Diversity–Fluency Trade-off

If our qualitative observations hold under quantitative scrutiny, they suggest that AR and MDLM are *complementary* rather than competing paradigms:

- **AR** excels when fluency and coherence are paramount (translation, summarization, code generation).
- **MDLM** may be preferable when diversity is valued (creative writing, data augmentation, brainstorming).

This complementarity mirrors the broader machine learning landscape, where autoregressive and diffusion approaches coexist in image generation, speech synthesis, and music generation.

5.4 Limitations

1. **Small scale.** Our models are 123–163M parameters trained on 50M tokens. At 1B+ scale, the dynamics may differ: AR overfitting may be less pronounced with more data, and MDLM may benefit from scale differently.
2. **Parameter mismatch.** The MDLM has 31.6% more parameters (Section 3.2). This is an inherent asymmetry, but it complicates attribution of observed differences.
3. **No quantitative diversity metrics.** Our diversity claim rests on qualitative inspection of a small number of samples. Self-BLEU, Distinct- n , MAUVE, and entropy-based metrics over ≥ 500 samples are necessary for rigorous validation.
4. **Single dataset.** TinyStories is a narrow domain (children’s stories). Generalization to code, scientific text, dialogue, and multilingual data is unknown.
5. **MDLM still converging.** The MDLM had not plateaued at 20K steps. Extended training could change the relative quality of generated text.
6. **Single seed.** All experiments use one random seed. Multiple seeds with confidence intervals would strengthen all claims.
7. **No downstream evaluation.** We do not evaluate on classification, QA, or other downstream tasks.
8. **Sampler not optimized.** The MDLM sampler (confidence-based unmasking, temperature annealing, repetition penalty) was not extensively tuned. Better sampling strategies (e.g., nucleus-based unmasking, classifier-free guidance for language) could improve MDLM output quality.

6 Conclusion

We present a controlled comparison between autoregressive and masked diffusion language models, trained on identical data, budget, and hardware. Our findings:

1. **Training throughput is near-identical:** MDLM trains at 95.5% of AR throughput, suggesting that diffusion training overhead is minimal at the Transformer scale.
2. **Convergence regimes differ:** AR converges in $\sim 14\text{K}$ steps then overfits; MDLM converges monotonically through 20K steps with no plateau, implying different optimal training lengths.
3. **Generation profiles differ qualitatively:** AR produces fluent but formulaic text; MDLM produces diverse but occasionally ungrammatical text. This preliminary observation suggests a diversity–fluency trade-off that, if confirmed, would imply the paradigms are complementary rather than competing.

We release all code, trained models, and data pipelines at <https://github.com/caiovicentino/arche>. Future work should (i) validate the diversity observation quantitatively, (ii) extend the comparison to larger scales and diverse datasets, and (iii) train MDLM for longer to determine its convergence characteristics.

References

- Austin, J., Johnson, D., Ho, J., Tarlow, D., & van den Berg, R. (2021). Structured Denoising Diffusion Models in Discrete State-Spaces. *NeurIPS 2021*. arXiv:2107.03006.
- DeepSeek-AI. (2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434.
- Eldan, R. & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? arXiv:2305.07759.
- Google DeepMind. (2025). Gemini Diffusion. Technical announcement, <https://deepmind.google/models/gemini-diffusion/>.
- Gu, A. & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.
- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *NeurIPS 2020*. arXiv:2006.11239.
- Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models. *NeurIPS 2022*. arXiv:2203.15556.
- Kaplan, J., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361.
- Li, J., Galley, M., Brockett, C., Gao, J., & Dolan, B. (2016). A Diversity-Promoting Objective Function for Neural Conversation Models. *NAACL-HLT 2016*.
- Lou, A., Meng, C., & Ermon, S. (2024). Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. *ICML 2024 Best Paper*. arXiv:2310.16834.
- Nie, S., et al. (2025). LLaDA: Large Language Diffusion with mAsking. arXiv:2502.09992.
- Pillutla, K., et al. (2021). MAUVE: Measuring the Gap Between Neural Text and Human Text using Divergence Frontiers. *NeurIPS 2021*. arXiv:2102.01454.

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI Technical Report.
- Sahoo, S., et al. (2024). Simple and Effective Masked Diffusion Language Models. *NeurIPS 2024*. arXiv:2406.07524.
- Song, Y., Sohl-Dickstein, J., Kingma, D.P., Kumar, A., Ermon, S., & Poole, B. (2021). Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR 2021*. arXiv:2011.13456.
- Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.
- Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762.
- Yang, S., et al. (2024). Gated Linear Attention Transformers with Hardware-Efficient Training. *ICML 2024*. arXiv:2312.06635.
- Zhu, Y., et al. (2018). Texygen: A Benchmarking Platform for Text Generation Models. *SIGIR 2018*. arXiv:1802.01886.

A MDLM Implementation Details

Cosine noise schedule. The masking probability follows $\gamma(t) = 1 - \cos^2(\pi t/2)$ for $t \in [0, 1]$. At $t = 0$, no tokens are masked (clean input); at $t = 1$, all tokens are masked (pure noise). During training, t is sampled uniformly for each example in the batch.

Confidence-based unmasking. During generation, at each of the $S = 100$ denoising steps, the model produces logits for all masked positions. We compute the maximum softmax probability at each masked position and unmask the top- k most confident positions, where $k = \lfloor L/S \rfloor$ and L is the sequence length. Ties are broken randomly.

Temperature annealing. We apply a linear temperature schedule from $\tau_{\text{start}} = 1.2$ to $\tau_{\text{end}} = 0.5$ across the S denoising steps. High temperature early in generation encourages diverse initial predictions; low temperature late in generation sharpens the distribution for coherent refinement.

Repetition penalty. We apply a frequency-based repetition penalty: for any token that has already been unmasked, we divide its logit by a penalty factor of 1.3. This reduces the tendency to produce degenerate repetitive sequences.

B Compute Resources

Table 4: Compute budget for all experiments reported in this paper.

Model	Hardware	Time	Cost
AR (123.6M, 20K steps)	H100 80GB (Colab)	107.9 min	~\$5
MDLM (162.7M, 20K steps)	H100 80GB (Colab)	113.0 min	~\$5
Total		220.9 min	~\$10

C Data

All experiments use 50M tokens from TinyStories [Eldan & Li, 2023], a dataset of short children’s stories generated by GPT-3.5 and GPT-4, specifically designed for training and evaluating small language models. The dataset was tokenized using the GPT-2 tokenizer (vocabulary size 50,257). We split the tokenized data into training and validation sets, using the last 5% of chunks as the validation set.